

프로그래밍응용개념

#05 참조 타입

국립한밭대학교 인공지능 소프트웨어학과
이성민

참조 타입

- 문자열 타입
- 배열 타입
- 열거 타입

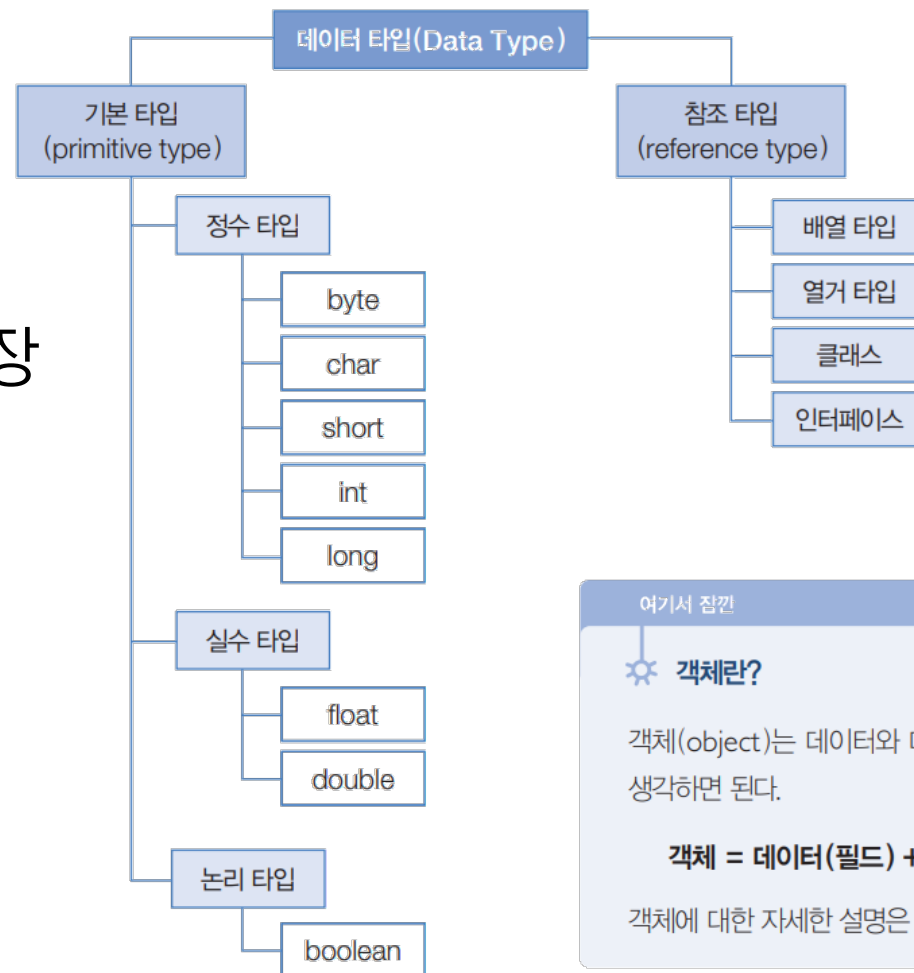
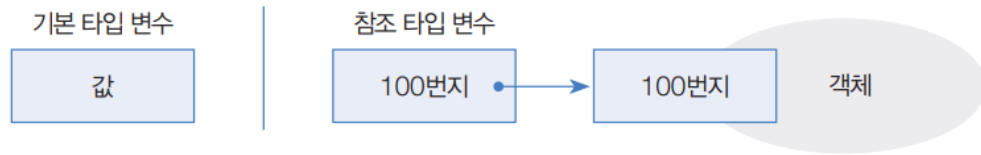
데이터 타입 분류

- 기본 타입

- 선언된 값 자체를 저장
- 정수, 실수, 논리 타입

- 참조 타입

- 객체가 생성된 메모리 주소를 저장
- 배열, 열거, 클래스, 인터페이스



여기서 잠깐

⚙ 객체란?

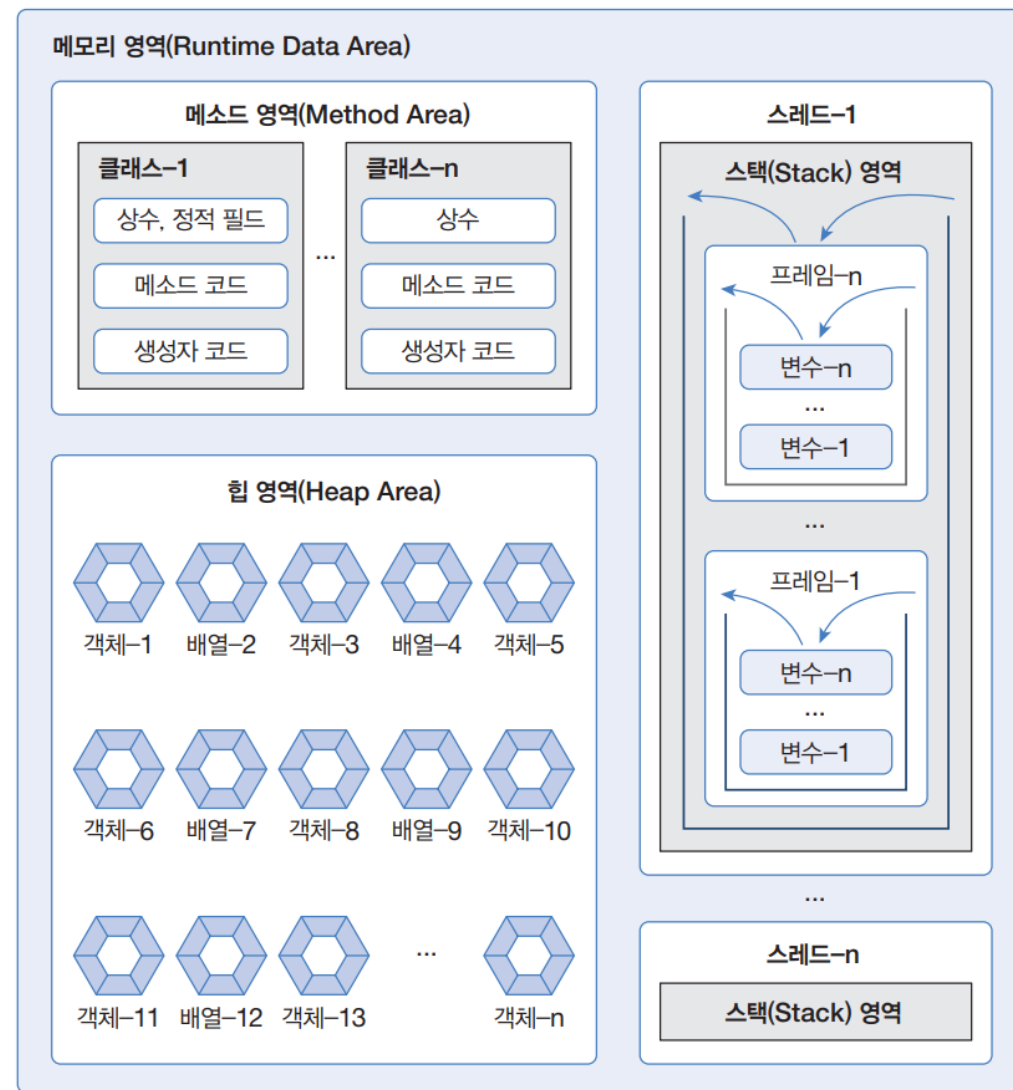
객체(object)는 데이터와 메소드로 구성된 덩어리라고 생각하면 된다.

객체 = 데이터(필드) + 메소드

객체에 대한 자세한 설명은 6장에서 살펴본다.

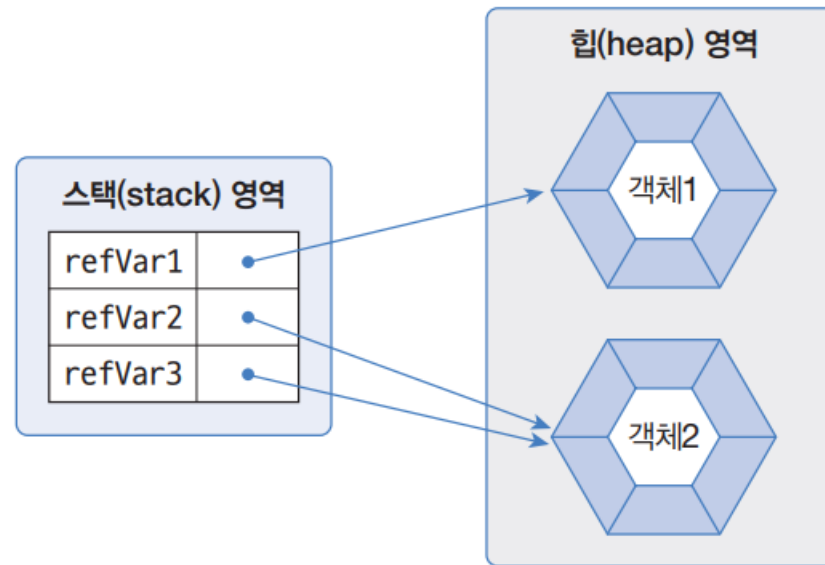
메모리 사용 영역

- 메소드, 힙, 스택 영역
 - 메소드 영역
 - 클래스 정보들이 저장되는 영역
 - 프로그램이 종료할 때 까지 메모리에 남음
 - 힙 영역
 - 객체와 배열이 생성되는 영역
 - 스택 영역
 - 기본 타입 변수들이 저장되는 영역
 - 메소드 마다 스택 프레임들을 가짐
 - 프레임: 메소드에 필요한 메모리 덩어리



참조 타입 연산

- ==, != 연산자
 - 객체의 주소를 비교해 변수의 값이 같은지 아닌지를 비교
 - 주소가 같다면, 동일한 객체를 참조하는 것
 - 주소가 다르다면, 다른 객체를 참조하는 것



참조 타입 연산

- ==, != 연산자

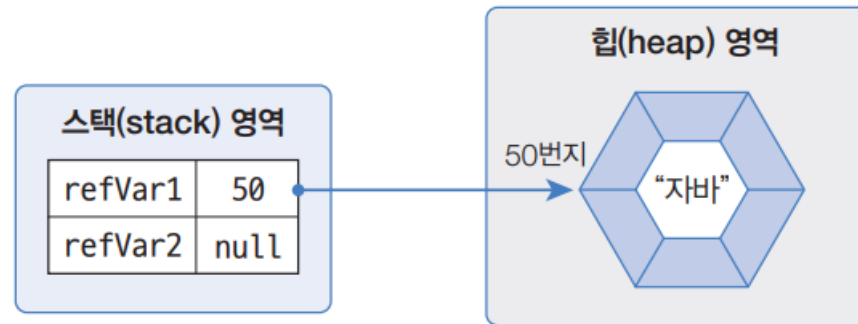
```
int[] arr1;           //배열 변수 arr1 선언
int[] arr2;           //배열 변수 arr2 선언
int[] arr3;           //배열 변수 arr3 선언

arr1 = new int[] { 1, 2, 3 }; //배열 { 1, 2, 3 }을 생성하고 arr1 변수에 대입
arr2 = new int[] { 1, 2, 3 }; //배열 { 1, 2, 3 }을 생성하고 arr2 변수에 대입
arr3 = arr2;            //배열 변수 arr2의 값을 배열 변수 arr3에 대입

System.out.println(arr1 == arr2); //arr1과 arr2 변수 값이 동일한 변지인지 검사
System.out.println(arr2 == arr3); //arr2와 arr3 변수가 같은 배열을 참조하는지 검사
```

참조 타입 연산

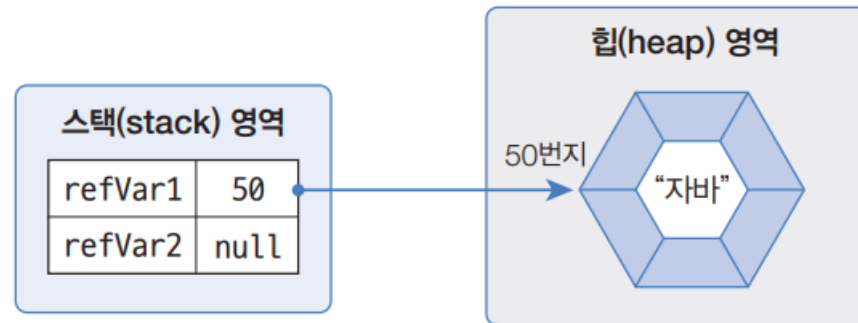
- null 값
 - null 값: 해당 참조 타입 변수는 아직 주소를 저장하지 않다는 뜻
 - null도 초기값으로 사용할 수 있기 때문에, null로 초기화 된 참조 변수는 스택 영역에 생성



참조 타입 연산

- NullPointerException

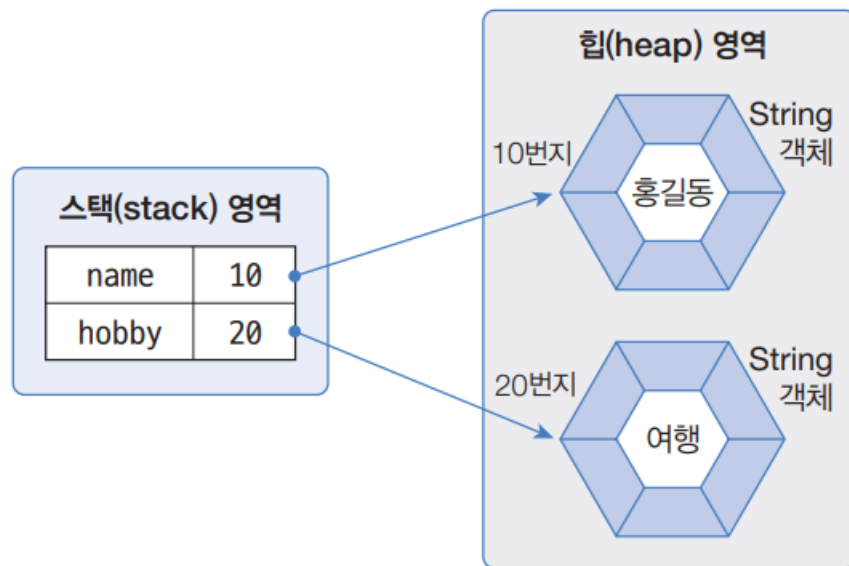
- 변수가 null인 상태에서 객체의 데이터나 메소드를 사용하려 할 때 발생하는 예외
- 참조 변수가 객체를 정확히 참조하도록 주소를 대입해야 해결됨



문자열 타입

- String 타입
 - 문자열을 String 객체로 생성

```
String name;           //String 타입 변수 name 선언  
name = "홍길동";       //name 변수에 문자열 대입  
String hobby = "여행"; //String 타입 변수 hobby를 선언하고 문자열 대입
```

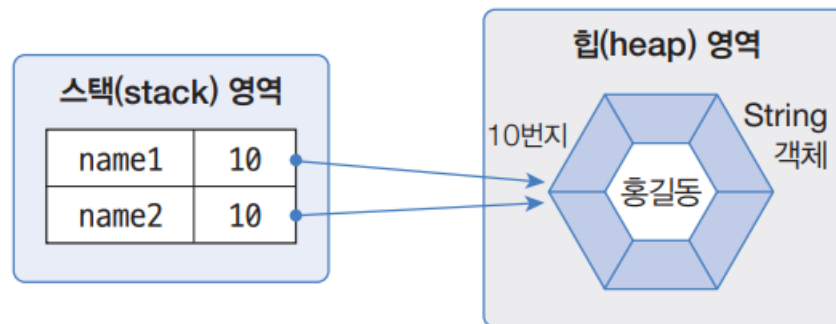


문자열 타입

- String 타입

- 문자열 리터럴이 동일하다면 String 객체를 공유
 - new 연산자로 직접 String 객체를 생성한다면, 다른 주소를 가지게 됨

```
String name1 = "홍길동";  
String name2 = "홍길동";
```



문자열 타입

- String 타입

- ==, != 은 해당 String이 가리키고 있는 주소가 같은지 비교
- .equals() 메소드는 문자열이 같은지 비교

```
String strVar1 = "홍길동";
String strVar2 = "홍길동";

if(strVar1 == strVar2) {
    System.out.println("strVar1과 strVar2는 참조가 같음");
} else {
    System.out.println("strVar1과 strVar2는 참조가 다름");
}

if(strVar1.equals(strVar2)) {
    System.out.println("strVar1과 strVar2는 문자열이 같음");
}
```

```
String strVar3 = new String( original: "홍길동");
String strVar4 = new String( original: "홍길동");

if(strVar3 == strVar4) {
    System.out.println("strVar3과 strVar4는 참조가 같음");
} else {
    System.out.println("strVar3과 strVar4는 참조가 다름");
}

if(strVar3.equals(strVar4)) {
    System.out.println("strVar3과 strVar4는 문자열이 같음");
}
```

문자열 타입

- 빈 문자열
 - "" 를 사용해 빈 문자열을 만들 수 있음

```
String hobby = "";  
if(hobby.equals("")) {  
    System.out.println("hobby 변수가 참조는 String 객체는 빈문자열");  
}
```

문자열 타입

- 문자열 추출

- charAt() 메소드로 문자열에서 매개값으로 주어진 인덱스의 문자를 반환해 특정 위치의 문자를 얻을 수 있음

자	바		프	로	그	래	밍
0	1	2	3	4	5	6	7

문자열 타입

- 문자열 추출

- charAt() 메소드로 문자열에서 매개값으로 주어진 인덱스의 문자를 반환해 특정 위치의 문자를 얻을 수 있음

```
String ssn = "010624-1230123";  
char sex = ssn.charAt(7);  
switch (sex) {  
    case '1':  
    case '3':  
        System.out.println("남자 입니다.");  
        break;  
    case '2':  
    case '4':  
        System.out.println("여자 입니다.");  
        break;  
}
```

문자열 타입

- 문자열 길이
 - length() 메소드를 사용



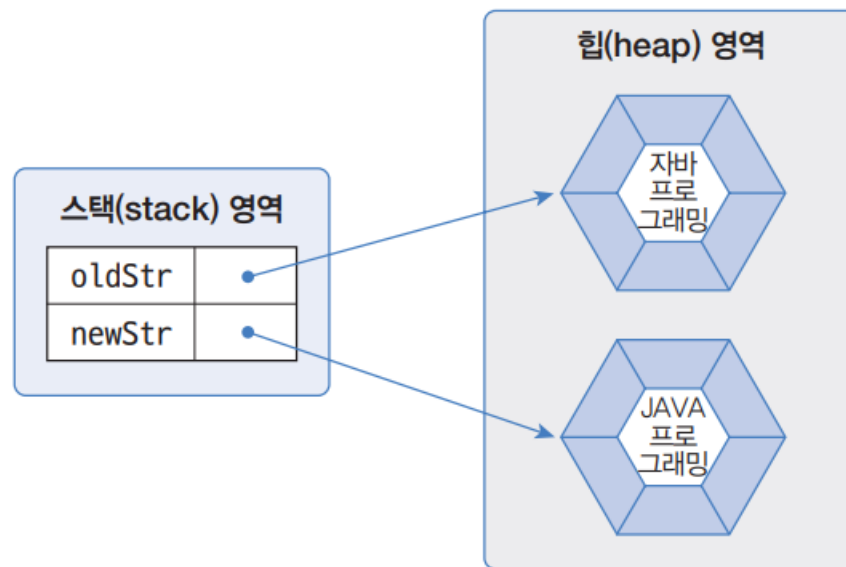
```
String ssn = "010624-1230123";
int length = ssn.length();
if(length == 14) {
    System.out.println("주민번호 자리수가 맞습니다.");
} else {
    System.out.println("주민번호 자리수가 틀립니다.");
}
```

문자열 타입

- 문자열 대체

- replace() 메소드는 기존 문자열을 그대로 두고, 대체한 새로운 문자열을 반환

```
String oldStr = "자바 프로그래밍";  
String newStr = oldStr.replace("자바", "JAVA");
```



문자열 타입

- 문자열 대체

- `replace()` 메소드는 기존 문자열을 그대로 두고, 대체한 새로운 문자열을 반환

```
String oldStr = "자바 문자열은 불변입니다. 자바 문자열은 String입니다.";
String newStr = oldStr.replace( target: "자바", replacement: "JAVA");

System.out.println(oldStr);
System.out.println(newStr);
```

```
String str1 = "안녕하세요 국립한밭대학교 인공지능소프트웨어학과입니다.";
String str2 = str1.replace( target: " ", replacement: "");
System.out.println(str2);
```

문자열 타입

- 문자열 잘라내기
 - substring() 메소드를 사용

메소드	설명
substring(int beginIndex)	beginIndex에서 끝까지 잘라내기
substring(int beginIndex, int endIndex)	beginIndex에서 endIndex 앞까지 잘라내기

```
String ssn = "880815-1234567 ";  
  
String firstNum = ssn.substring(0, 6);  
System.out.println(firstNum);  
  
String secondNum = ssn.substring(beginIndex: 7);  
System.out.println(secondNum);
```

문자열 타입

- 문자열 찾기

- indexOf() 메소드 사용
 - 해당 문자열이 시작되는 인덱스를 반환
 - 만약 해당 문자열이 없다면, -1 반환

```
String subject = "자바 프로그래밍";  
int index = subject.indexOf("프로그래밍");
```

- 문자열 포함

- contains() 메소드 사용
 - 해당 문자열 포함 여부에 따라 true/false 반환

자	바		프	로	그	래	밍
0	1	2	3	4	5	6	7

문자열 타입

- 문자열 찾기

```
String subject = "자바 프로그래밍";

int location = subject.indexOf("프로그래밍");
System.out.println(location);
String substring = subject.substring(location);
System.out.println(substring);

location = subject.indexOf("자바");
if(location != -1) {
    System.out.println("자바와 관련된 책이군요");
} else {
    System.out.println("자바와 관련없는 책이군요");
}

boolean result = subject.contains("자바");
if(result) {
    System.out.println("자바와 관련된 책이군요");
} else {
    System.out.println("자바와 관련없는 책이군요");
}
```

문자열 타입

- 문자열 분리
 - split() 메소드 사용
 - 해당 구분자를 기준으로 문자열을 분리

```
String board = "번호,제목,내용,글쓴이";  
String[] arr = board.split(",");
```

arr[0]	arr[1]	arr[2]	arr[3]
"번호"	"제목"	"내용"	"성명"

문자열 타입

- 문자열 분리

```
String board = "1,자바학습,참조타입 String을 학습합니다.,홍길동";

//문자열 분리
String[] tokens = board.split(regex: ",");

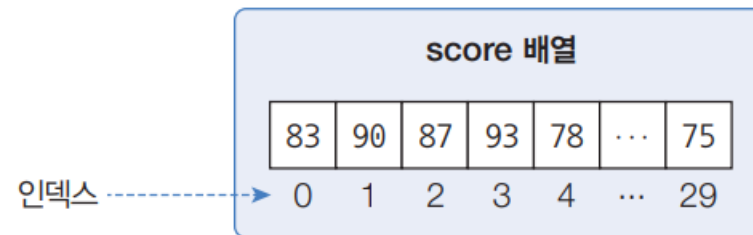
//인덱스별로 읽기
System.out.println("번호: " + tokens[0]);
System.out.println("제목: " + tokens[1]);
System.out.println("내용: " + tokens[2]);
System.out.println("쓴이: " + tokens[3]);
System.out.println();

//for문을 이용한 읽기
for(int i=0; i<tokens.length; i++) {
    System.out.println(tokens[i]);
}
```

배열 타입

- 배열

- 연속된 공간에 값을 나열하고, 각 값에 인덱스를 부여해 놓은 자료 구조
- 인덱스는 대괄호 []와 함께 사용하여 각 항목의 값을 읽거나 저장



score[0] = 83
score[1] = 90
score[2] = 87
score[3] = 93
score[29] = 75

배열 타입

- 배열 변수 선언
 - 두가지 형태로 작성 가능, 첫 번째가 일반적

```
타입[] 변수;
```

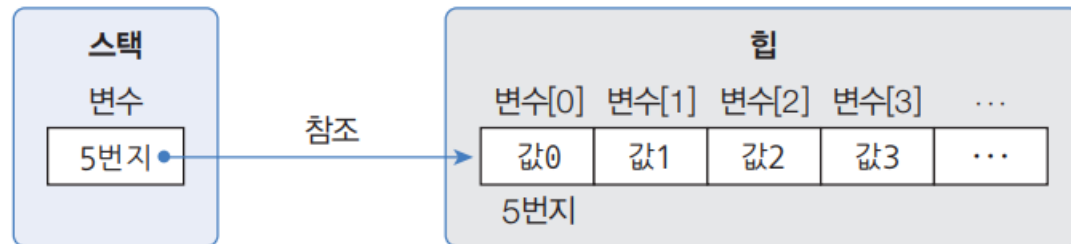
```
타입 변수[];
```


배열 타입

- 배열 변수 선언

- 배열은 힙 영역에 생성되고, 배열 변수는 스택 영역에 생성되어 힙 영역의 배열 주소를 저장
- 참조할 배열이 없다면, 배열 변수도 null로 초기화 가능

```
타입[] 변수 = { 값0, 값1, 값2, 값3, ... };
```



배열 타입

- 배열 변수 선언

- 배열은 힙 영역에 생성되고, 배열 변수는 스택 영역에 생성되어 힙 영역의 배열 주소를 저장

```
//배열 변수 선언과 배열 생성
String[] season = { "Spring", "Summer", "Fall", "Winter" };

//배열의 항목값 읽기
System.out.println("season[0] : " + season[0]);
System.out.println("season[1] : " + season[1]);
System.out.println("season[2] : " + season[2]);
System.out.println("season[3] : " + season[3]);

//인덱스 1 항목의 값 변경
season[1] = "여름";
System.out.println("season[1] : " + season[1]);
System.out.println();
```

```
//배열 변수 선언과 배열 생성
int[] scores = { 83, 90, 87 };

//총합과 평균 구하기
int sum = 0;
for(int i=0; i<3; i++) {
    sum += scores[i];
}

System.out.println("총합 : " + sum);
double avg = (double) sum / 3;
System.out.println("평균 : " + avg);
```

배열 타입

- 배열 길이

- 배열의 길이란 배열에 저장할 수 있는 항목의 수
- 배열의 길이를 얻으려면 배열의 length 필드를 읽음

```
배열변수.length;
```

- length 필드는 읽기만 가능하고, 값 변경 불가
- 배열에 for문을 적용할 때 주로 사용

배열 타입

- 배열 길이

```
//배열 변수 선언과 배열 대입
int[] scores = { 84, 90, 96 };

//배열 항목의 총합 구하기
int sum = 0;
for(int i=0; i<scores.length; i++) {
    sum += scores[i];
}
System.out.println("총합 : " + sum);

//배열 항목의 평균 구하기
double avg = (double) sum / scores.length;
System.out.println("평균 : " + avg);
```

배열 타입

- 배열 변수 선언
 - 배열 변수를 선언한 시점과 값이 대입되는 시점이 다르다면, new 연산자와 대괄호 []를 사용

```
변수 = new 타입[] { 값0, 값1, 값2, 값3, ... };
```

배열 타입

- 배열 변수 선언
 - 배열 변수를 선언한 시점과 값이 대입되는 시점이 다르다면, new 연산자와 대괄호 []를 사용

```
//배열 변수 선언
int[] scores;
//배열 변수에 배열을 대입
scores = new int[] { 83, 90, 87 };
//배열 항목의 총합을 구하고 출력
int sum1 = 0;
for(int i=0; i<3; i++) {
    sum1 += scores[i];
}
System.out.println("총합 : " + sum1);
```

```
int[] scores;

scores = new int[] {83, 90, 87};

int maxNum = 0;
for (int i=0; i<scores.length; i++) {
    if (scores[i] > maxNum) {
        maxNum = scores[i];
    }
}
System.out.println("최댓값: " + maxNum);
```

배열 타입

- new 연산자로 배열 생성
 - new 연산자로 값은 없지만 향후 값들을 저장할 목적으로 배열 미리 생성
 - 길이를 지정해 줘야 함

```
타입[] 변수 = new 타입[길이];
```

배열 타입

- new 연산자로 배열 생성
 - 이 경우에는 배열은 다음의 기본값으로 초기화 됨

데이터 타입		초기값
기본 타입	byte[]	0
	char[]	'\u0000'
	short[]	0
	int[]	0
	long[]	0L
	float[]	0.0F
참조 타입	double[]	0.0
	boolean[]	false
	클래스[]	null
	인터페이스[]	null

배열 타입

- new 연산자로 배열 생성

```
int[] arr1 = new int[3];  
for (int i=0; i<arr1.length; i++) {  
    System.out.println(arr1[i]);  
}  
  
arr1[0] = 10;  
arr1[1] = 20;  
arr1[2] = 30;  
for (int i: arr1) {  
    System.out.println(i);  
}
```

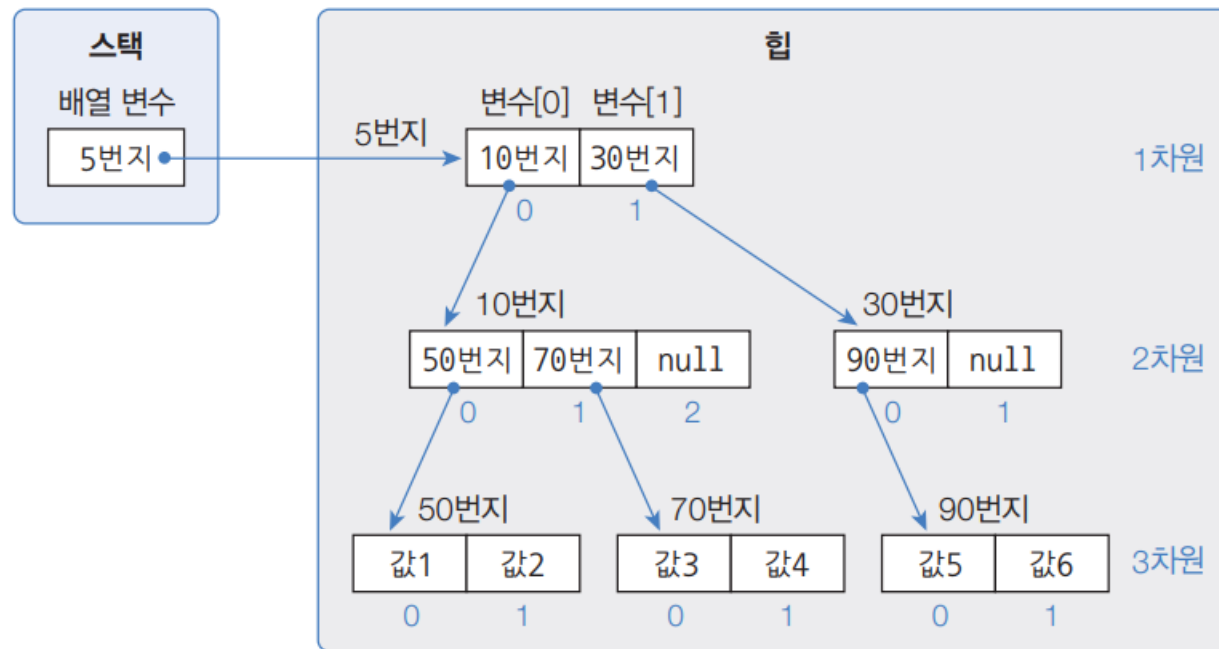
배열 타입

- new 연산자로 배열 생성

```
for (int i=0; i<arr2.length; i++) {  
    System.out.println(arr2[i]);  
}  
arr2[0] = "1월";  
arr2[1] = "2월";  
arr2[2] = "3월";  
for (String s: arr2) {  
    System.out.println(s);  
}
```

배열 타입

- 다차원 배열
 - 배열 내부에 또 다른 배열이 대입된 배열

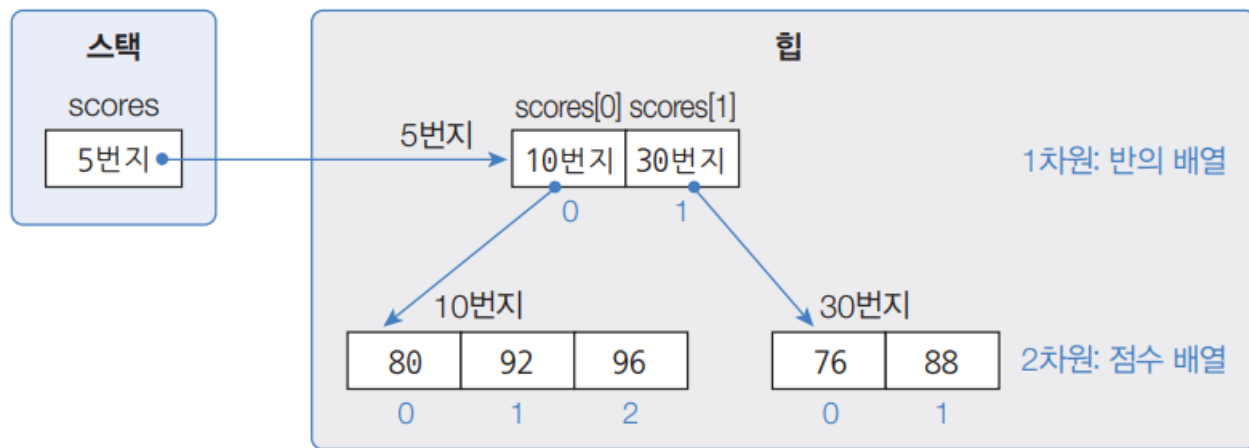


변수[1차원인덱스][2차원인덱스]...[N차원인덱스]

배열 타입

- 값을 사용한 다차원 배열 생성

```
타입[][] 변수 = {  
    {값1, 값2, ...},      1차원 배열의 0 인덱스  
    {값3, 값4, ...},      1차원 배열의 1 인덱스  
    ...  
};
```



배열 타입

- 값을 사용한 다차원 배열 생성

```
//2차원 배열 생성
int[][] scores = {
    { 80, 90, 96 },
    { 76, 88 }
};

//배열의 길이
System.out.println("1차원 배열 길이(반의 수): " + scores.length);
System.out.println("2차원 배열 길이(첫번째 반의 학생 수): " + scores[0].length);
System.out.println("2차원 배열 길이(두번째 반의 학생 수): " + scores[1].length);

//첫번째 반의 세번째 학생의 점수 읽기
System.out.println("scores[0][2]: " + scores[0][2]);

//두번째 반의 두번째 학생의 점수 읽기
System.out.println("scores[1][1]: " + scores[1][1]);

//첫번째 반의 평균 점수 구하기
int class1Sum = 0;
for(int i=0; i<scores[0].length; i++) {
    class1Sum += scores[0][i];
}
double class1Avg = (double) class1Sum / scores[0].length;
System.out.println("첫번째 반의 평균 점수: " + class1Avg);
```

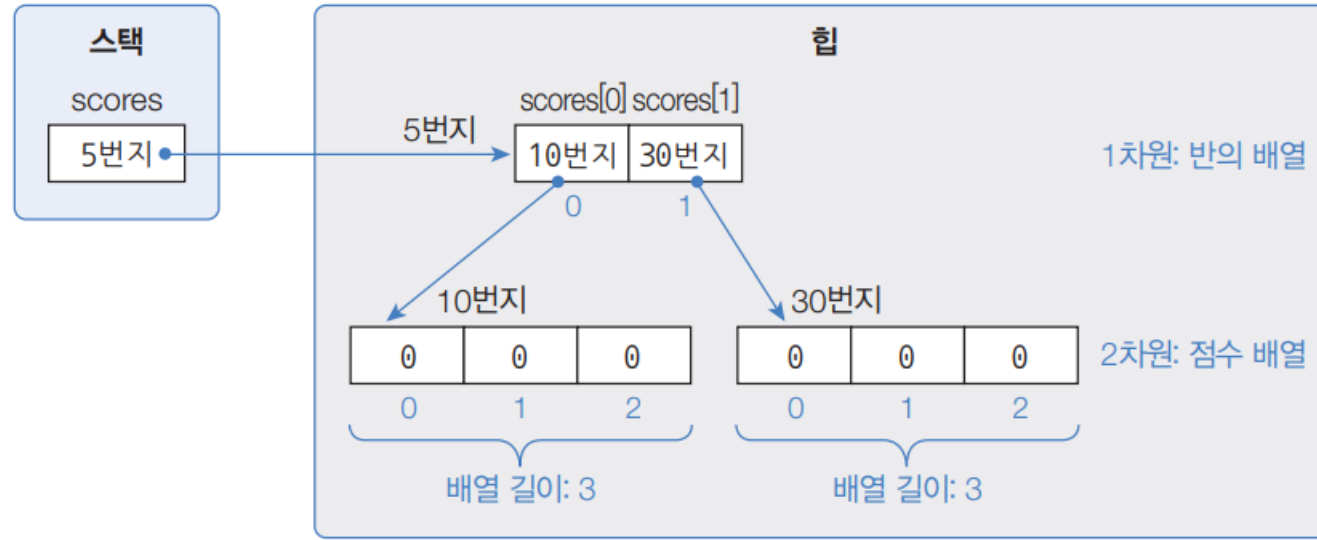
```
//두번째 반의 평균 점수 구하기
int class2Sum = 0;
for(int i=0; i<scores[1].length; i++) {
    class2Sum += scores[1][i];
}
double class2Avg = (double) class2Sum / scores[1].length;
System.out.println("두번째 반의 평균 점수: " + class2Avg);

//전체 학생의 평균 점수 구하기
int totalStudent = 0;
int totalSum = 0;
for(int i=0; i<scores.length; i++) {
    totalStudent += scores[i].length;
    for(int k=0; k<scores[i].length; k++) {
        totalSum += scores[i][k];
    }
}
double totalAvg = (double) totalSum / totalStudent;
System.out.println("전체 학생의 평균 점수: " + totalAvg);
```

배열 타입

- new 연산자를 사용한 다차원 배열 생성
 - 배열의 길이 (차원수) 명시 필요

```
타입[][] 변수 = new 타입[1차원수][2차원수];
```



배열 타입

- new 연산자를 사용한 다차원 배열 생성
 - 배열의 크기가 일정할 경우

```
// 각 반의 학생 수가 3명으로 동일할 경우 점수 저장을 위한 2차원 배열 생성
int[][] mathScores = new int[2][3];
// 배열 항목 초기값 출력
for (int i = 0; i < mathScores.length; i++) {           // 반의 수만큼 반복
    for (int k = 0; k < mathScores[i].length; k++) {    // 해당 반의 학생 수만큼 반복
        System.out.println("mathScores[" + i + "][" + k + "]: " + mathScores[i][k]);
    }
}
System.out.println();
// 배열 항목 값 변경
mathScores[0][0] = 80;
mathScores[0][1] = 83;
mathScores[0][2] = 85;
mathScores[1][0] = 86;
mathScores[1][1] = 90;
mathScores[1][2] = 92;
```

배열 타입

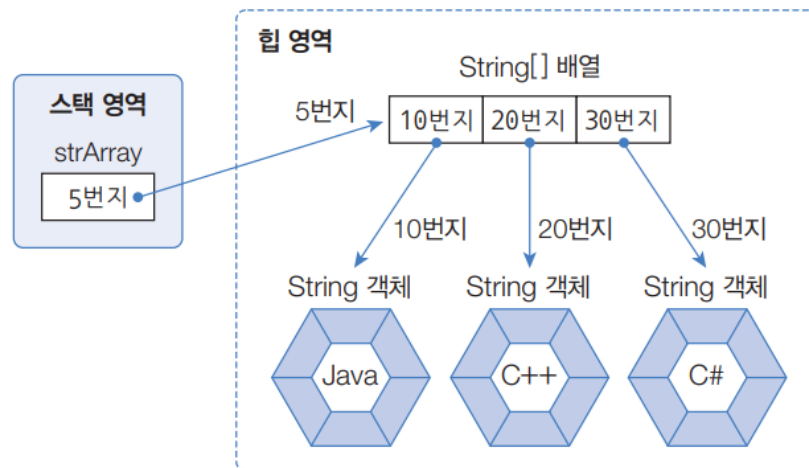
- new 연산자를 사용한 다차원 배열 생성
 - 배열의 크기가 일정 하지 않을 경우

```
// 각 반의 학생 수가 다를 경우 점수 저장을 위한 2차원 배열 생성
int[][] englishScores = new int[2][];
englishScores[0] = new int[2];
englishScores[1] = new int[3];
// 배열 항목 초기값 출력
for (int i = 0; i < englishScores.length; i++) {           // 반의 수만큼 반복
    for (int k = 0; k < englishScores[i].length; k++) { // 해당 반의 학생 수만큼 반복
        System.out.println("englishScores[" + i + "][" + k + "]: " + englishScores[i][k]);
    }
}
System.out.println();
// 배열 항목 값 변경
englishScores[0][0] = 90;
englishScores[0][1] = 91;
englishScores[1][0] = 92;
englishScores[1][1] = 93;
englishScores[1][2] = 94;
```


배열 타입

- 객체를 참조하는 배열
 - 기본 타입 배열은 각 항목에 값을 직접 저장
 - 참조 타입 (클래스, 인터페이스) 배열은 각 항목에 객체의 주소를 저장

```
String[] strArray = new String[3];  
strArray[0] = "Java";  
strArray[1] = "C++";  
strArray[2] = "C#";
```



배열 타입

- 객체를 참조하는 배열
 - 기본 타입 배열은 각 항목에 값을 직접 저장
 - 참조 타입 (클래스, 인터페이스) 배열은 각 항목에 객체의 주소를 저장

```
String[] strArray = new String[3];
strArray[0] = "Java";
strArray[1] = "Java";
strArray[2] = new String( original: "Java");

System.out.println( strArray[0] == strArray[1] );    //true: 같은 객체 참조
System.out.println( strArray[0] == strArray[2] );    //false: 다른 객체를 참조
System.out.println( strArray[0].equals(strArray[2]) ); //true: 문자열이 동일
```

배열 타입

- 배열 복사

- 배열은 한 번 생성하면 길이를 변경할 수 없음
- 더 많은 저장공간이 필요하다면 더 큰 길이의 배열을 새로 만들고, 이전 배열로부터 값을 복사해야 함



배열 타입

- 배열 복사

- 배열은 한 번 생성하면 길이를 변경할 수 없음
- 더 많은 저장공간이 필요하다면 더 큰 길이의 배열을 새로 만들고, 이전 배열로부터 값을 복사해야 함

```
//길이 3인 배열
int[] oldIntArray = { 1, 2, 3 };
//길이 5인 배열을 새로 생성
int[] newIntArray = new int[5];
//배열 항목 복사
for(int i=0; i<oldIntArray.length; i++) {
    newIntArray[i] = oldIntArray[i];
}
```

배열 타입

- 배열 복사

- System의 arraycopy() 메소드를 이용해 배열 복사 가능

```
System.arraycopy(Object src, int srcPos, Object dest, int destPos, int length);
```

↑
원본 배열

↑
원본 배열
복사
시작 인덱스

↑
새 배열

↑
새 배열
붙여넣기
시작 인덱스

↑
복사 항목 수

배열 타입

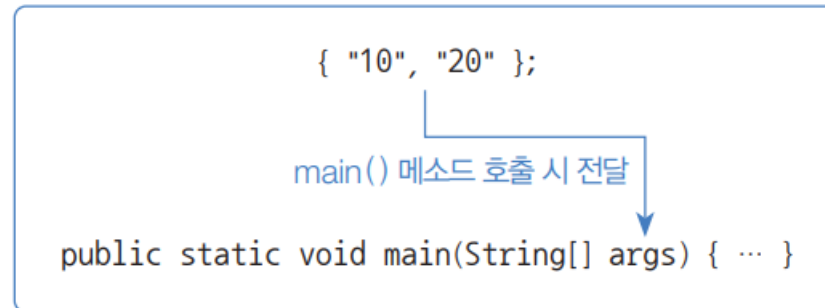
- 배열 복사

- System의 arraycopy() 메소드를 이용해 배열 복사 가능

```
//길이 3인 배열
String[] oldStrArray = { "java", "array", "copy" };
//길이 5인 배열을 새로 생성
String[] newStrArray = new String[5];
//배열 항목 복사
System.arraycopy( oldStrArray, srcPos: 0, newStrArray, destPos: 0, oldStrArray.length);
//배열 항목 출력
for(int i=0; i<newStrArray.length; i++) {
    System.out.print(newStrArray[i] + ", ");
}
```

배열 타입

- String[] args 매개변수
 - 자바 프로그램을 실행하기 위한 main() 메소드에는 문자열 배열인 String[] args 매개변수가 필요



배열 타입

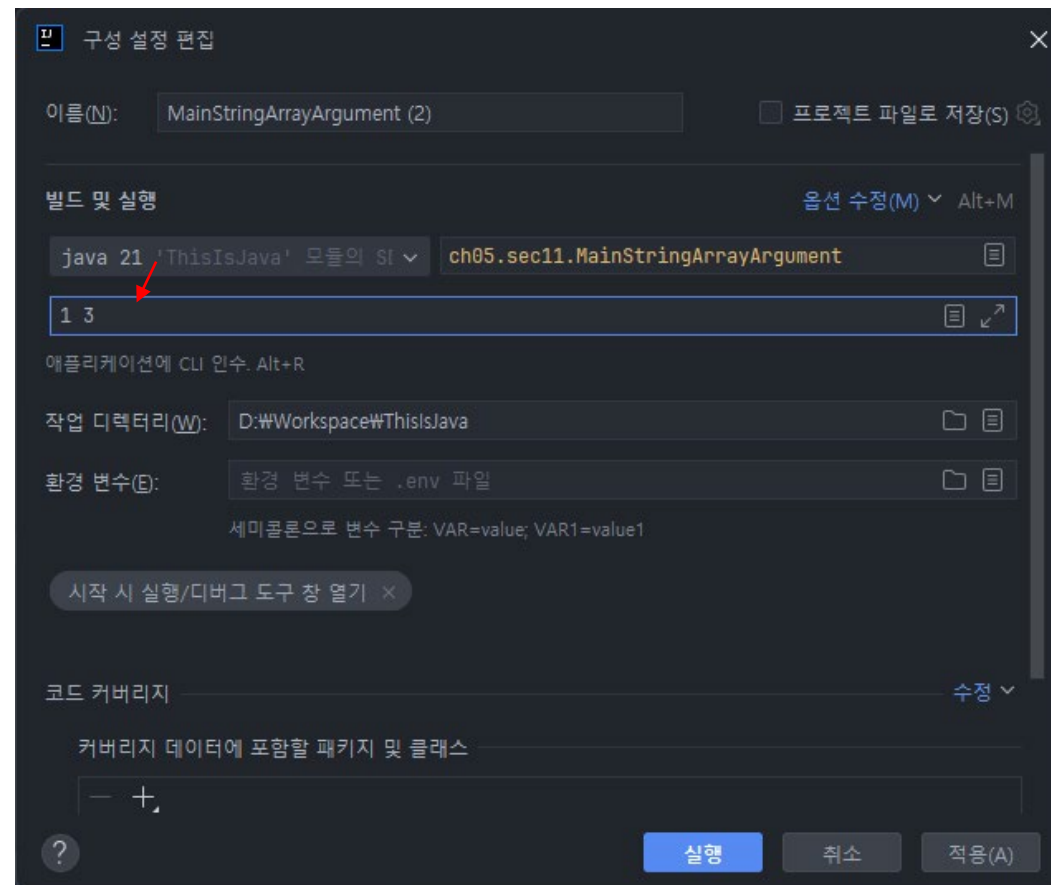
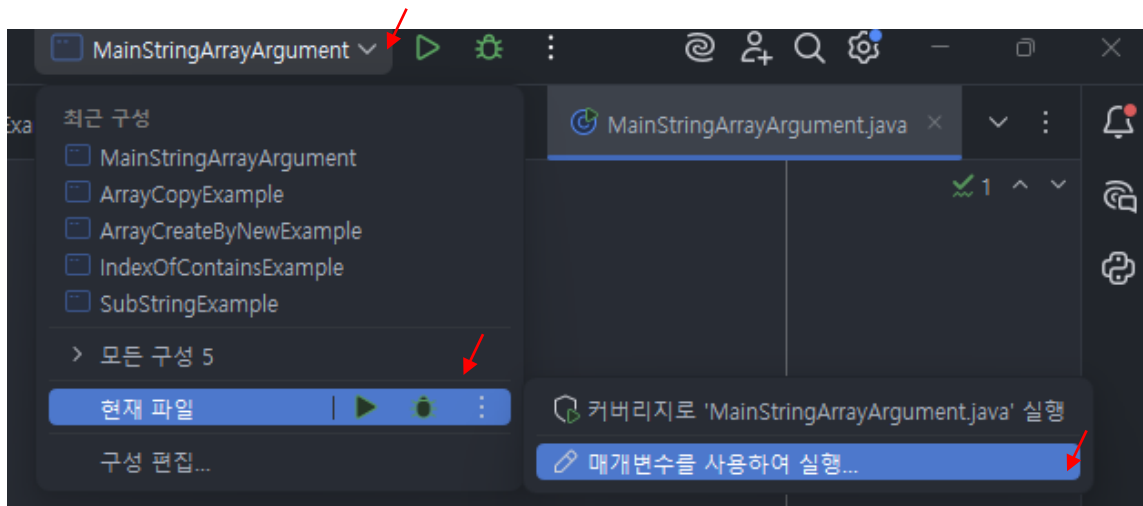
- String[] args 매개변수

```
public static void main(String[] args) {  
    if(args.length != 2) {  
        System.out.println("프로그램 입력 값이 부족");  
        System.exit(status: 0);  
    }  
  
    String strNum1 = args[0];  
    String strNum2 = args[1];  
  
    int num1 = Integer.parseInt(strNum1);  
    int num2 = Integer.parseInt(strNum2);  
  
    int result = num1 + num2;  
    System.out.println(num1 + " + " + num2 + " = " + result);  
}
```

```
(base) PS D:\Workspace\ThisIsJava> java .\src\ch05\sec11\MainStringArrayArgument.java 1 3  
1 + 3 = 4
```


배열 타입

- String[] args 매개변수



열거 타입

- Enum 타입

- 요일, 계절 등과 같이 특별하게 한정된 값을 정의하기 위한 타입
- 열거 타입의 이름은 첫 문자를 대문자로 하는 것이 일반적
- 값은 모두 대문자로 작성하고, 두 단어 이상일 경우 _ 로 연결

```
>>> Week.java

1  package ch05.sec12;
2
3  public enum Week {
4      MONDAY,
5      TUESDAY,
6      WEDNESDAY,
7      THURSDAY,
8      FRIDAY,
9      SATURDAY,
10     SUNDAY
11 }

    열거 타입 이름
    열거 상수 목록(한정된 값 목록)
```

열거 타입

- Enum 타입

```
public enum SeatStatus {EMPTY, RESERVED, BLOCKED} 8개 사용 위치

public static void main(String[] args) {
    SeatStatus[] seatStatuses = new SeatStatus[5];

    seatStatuses[0] = SeatStatus.EMPTY;
    seatStatuses[1] = SeatStatus.BLOCKED;
    seatStatuses[2] = SeatStatus.RESERVED;
    seatStatuses[3] = SeatStatus.EMPTY;
    seatStatuses[4] = SeatStatus.RESERVED;

    for (SeatStatus s: seatStatuses) {
        System.out.println(s);
    }
}
```

